

Programación con Objetos 3

Closures

Autores:

→ Facundo Javier Gelatti — javiergelatti@gmail.com

Este documento está bajo una licencia [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)



Índice

Closures	1
El concepto desde la programación funcional.....	1
Los Procs en Ruby son closures.....	2
Otras ligaduras posibles.....	3
Para las personas curiosas.....	5

Closures

El objetivo de este apunte es repasar el concepto de *closure* en programación, con algunos ejemplos en Ruby. Si buscan en la web, van a encontrarse con muchas definiciones relacionadas (y a veces contrapuestas) de lo que es un *closure*. No hay tanta variedad como en el caso de la definición de “objeto”; pero amerita que tengamos material específico.

El concepto desde la programación funcional

La idea de *closure* viene de la programación funcional. El término fue usado por primera vez en el artículo *The mechanical evaluation of expressions* (Landin, 1964) para describir una manera de representar *el valor de una expresión lambda*¹:

[...] we represent the value of a λ -expression by a bundle of information called a “closure,” comprising the λ -expression and the environment relative to which it was evaluated.

Es decir, un *closure* se compone de una expresión lambda y un contexto de ejecución. Veamos mejor en qué consiste esto.

En lo que sigue voy a usar sintaxis de pseudo-código basado en el *cálculo lambda* para la definición de las funciones, pero me voy a tomar la libertad de dejar los envíos de mensajes con la sintaxis de Ruby. Por ejemplo:

$\lambda x. x + y$ conceptualmente equivale a `proc { |x| x + y }`

El nombre “closure” tiene que ver con el vocabulario técnico que se usa en programación funcional, a saber:

- Se dice que una expresión lambda es *abierta* cuando contiene *variables libres* (es decir, variables que no están ligadas como parámetros²). Por ejemplo: $\lambda x. x + y$
- Por otro lado, una expresión lambda es *cerrada* cuando todas sus variables están ligadas, es decir, es autocontenida. Por ejemplo: $\lambda x. x + x$
- Cerrar* una expresión posiblemente abierta consiste en proporcionarle un contexto que provea ligaduras para todas sus variables libres. Por ejemplo, para la expresión en (a) podría ser $\{y \mapsto 2\}$.

Operacionalmente, un *closure* puede verse como un par (expresión- λ , contexto), en donde el contexto se asegura de que la expresión- λ quede cerrada. Por ejemplo:

$(\lambda x. x * z + y, \{y \mapsto 3, z \mapsto 4\})$

Esto permite que se pueda evaluar la expresión lambda simplemente proporcionando un valor para sus parámetros. En esto consiste *aplicar* el closure.

¹ No hay que confundir esto con el significado que se le da a la palabra “closure” en álgebra, que es otro completamente distinto.

² Esta es la única manera de definir variables en cálculo lambda.

Para ir transicionando hacia el uso de objetos, en lugar de ver a los closures como un mero *par*, podríamos verlos como objetos:

```
let un_closure = Closure.new(  $\lambda x. x * z + y$  , {  $y \mapsto 3, z \mapsto 4$  } )
```

Para luego poder mandarles mensajes:

```
un_closure.apply(2)  $\Rightarrow$  11
```

De todos modos, la idea no es que haya que crear un closure así “a mano”, sino que se cree implícitamente, de manera automática, al momento de evaluar una expresión lambda. El closure sería *el valor* de la expresión lambda, y contiene la expresión lambda que se evaluó y el contexto actual, en el que se está realizando la evaluación al momento de crear el closure. A este contexto se lo conoce como “contexto léxico”. Por ejemplo:

let y = 2 + 1	Configuramos un contexto inicial. La variable y se liga al resultado de evaluar la expresión “2 + 1”, que resulta en 3.
let z = 4	

let f = ($\lambda x. x * z + y$)	La variable f se liga al resultado de evaluar la expresión lambda, que resulta en un closure ($\lambda x. x * z + y$, { $y \mapsto 3, z \mapsto 4$ }). El significado de “y” y “z” se toma del contexto actual (esto es, <i>se ligan léxicamente</i>).
---	--

...

f.apply(2)	Más tarde, al aplicar el closure, se evalúa la expresión lambda ligando su parámetro (x) al argumento que se le pasa (2), y el resto de sus variables libres según el contexto asociado al closure, resultando en 11.
------------	---

Los Procs en Ruby son closures

En Ruby, las instancias de la clase `Proc` son *closures*. Al crearse, conocen el bloque de código ejecutable y también el contexto en el cuál se crearon. Podemos reproducir el mismo ejemplo anterior:

```
y = 2 + 1
z = 4

f = proc { |x| x * z + y }

f.call(2) # => 11
```

Para acceder al contexto asociado, podemos usar el mensaje `binding`. Este mensaje forma parte de la interfaz de reflexión, y retorna un objeto que representa ese contexto:

```
f.binding.local_variables # => [:y, :z, :f]
f.binding.local_variable_get(:y) # => 3
```

Por supuesto, el poder de los closures se ve cuando los creamos en un contexto y los ejecutamos en otro. Por ejemplo:

```
def crear_contador(valor_inicial = 0)
  cuenta = valor_inicial
  contar = proc { cuenta += 1 }
  actual = proc { cuenta }
  [contar, actual]
end

contar, actual = crear_contador
contar.call      # => 1
contar.call      # => 2
actual.call      # => 2
```

En el ejemplo, ambos procs tienen una referencia al mismo contexto porque ambos fueron creados durante la ejecución de `crear_contador`:

```
contar.binding.local_variables
# => [:valor_inicial, :cuenta, :contar, :actual]
actual.binding.local_variables
# => [:valor_inicial, :cuenta, :contar, :actual]
actual.binding.local_variable_get(:cuenta) # => 2
```

Podemos darnos cuenta de que comparten el mismo contexto porque hacer un cambio a partir de uno impacta al otro. Ya vimos esto en la primera ejecución, pero podemos hacerlo a través de la interfaz de reflexión:

```
actual.binding.local_variable_set(:cuenta, 999)
contar.call      # => 1000
```

Además, como cada activación del método `crear_contador` crea un contexto nuevo, podemos tener dos contadores independientes:

```
contar_1, _ = crear_contador(10)
contar_2, _ = crear_contador(5)
contar_1.call  # => 11
contar_1.call  # => 12
contar_2.call  # => 6
```

Otras ligaduras posibles

La definición clásica de closure, en particular, la manera de entender al contexto asociado a un closure, se puede extender más allá de los valores asociados a las variables. Por

ejemplo, el contexto capturado por un `Proc` incluye también a `self` (y, a través de `self`, las variables de instancia, etc.). Por eso podemos hacer cosas como esta:

```
class Persona
  def initialize(nombre)
    @nombre = nombre
  end
  def saludador
    proc { "Hola, soy #{@nombre}" }
  end
end

Persona.new("Marta").saludador.call # => "Hola, soy Marta"
```

Otra cosa que toman los `procs` de su contexto léxico es el `return`, que se liga de manera “no local”. Es decir, captura también el contexto de retorno de su método de origen. A este tipo de closures, que ligan también el `return`, se los suele llamar “*full closures*”. Por ejemplo:

```
def m1
  f = proc { return 10 }
  f.call
  15 + 5
end

m1 # => 10 (el 15 + 5 nunca se ejecuta)
```



Nota: Ruby ofrece también una alternativa que *no liga el return*, usando `lambda` en lugar de `proc`.

Si se intenta retornar de una activación de método que ya retornó, se levanta una excepción `LocalJumpError`. Por ejemplo:

```
def m1
  proc { return 123 } # retorna un proc que retorna... 🤖
end

m1.call # LocalJumpError: unexpected return
```

Esta manera de ligar el `return` nos sirve, por ejemplo, para poder implementar estructuras de control sin que sean primitivas del lenguaje, imitando la manera en la que funcionan los lenguajes imperativos. Pensemos por ejemplo en un `if` tradicional con un `return`:

```
def buscar
  if encontrado?
    return :ok
  end

  :nada
end
```

Claramente, si se ejecuta `return :ok`, esperamos que retorne el símbolo `:ok` del método `buscar`. Podemos, entonces, implementar el `if` usando objetos y mensajes (y, en particular, procs para reificar los bloques de código), manteniendo este mismo comportamiento:

```
class TrueClass
  def if_true(&bloque)
    bloque.call
  end
end

class FalseClass
  def if_true(&bloque)
    # no hacemos nada
  end
end

def buscar
  encontrado?.if_true do
    return :ok
  end

  :nada
end
```

Para las personas curiosas

El “retorno no local” también permite implementar de forma no primitiva cosas como el manejo de excepciones, que posibilita programar todo el mecanismo de excepciones “como una biblioteca”, sin que el lenguaje lo traiga incorporado.

La decisión de ligar `self` y `return` de manera léxica está relacionada también con el *principio de correspondencia de Tennent* (1981). Hubo discusiones sobre esto cuando se

estaba decidiendo la implementación de lambdas en Java³ y en JavaScript⁴, aunque finalmente ninguno de los dos lenguajes terminó teniendo *full closures*.

Pueden encontrar un resumen de la discusión histórica sobre el significado de “closure” (motivado por el desarrollo de una propuesta para su implementación en Java) y de qué cosas se pueden ligar léxicamente en el artículo *A Definition of Closures* de Neal Gafter (2007)⁵.

³ Por ejemplo,

<https://gafter.blogspot.com/2006/08/tennants-correspondence-principle-and.html>

<https://web.archive.org/web/20250213083851/http://javac.info/>

⁴ Por ejemplo,

<https://web.archive.org/web/20081015125619/http://wiki.ecmascript.org/doku.php?id=strawman:lambda>

⁵ <https://gafter.blogspot.com/2007/01/definition-of-closures.html>

Referencias

Landin, P. J. (1964). The mechanical evaluation of expressions. *The Computer Journal*, 4(6), 308-320. 10.1093/comjnl/6.4.308

Tennent, R. D. (1981). *Principles of programming languages*. Prentice/Hall International.